20-Day SQL Curriculum

for Data Analysts

Complete 18-day structured topic guide with order of execution

---

Phase 1 **Foundation** Days 1–5

Day 1 — Database fundamentals & SQL environment

What SQL is, how databases work, the query engine

Order of execution

`Environment setup`

Core concepts

- Relational model & RDBMS
- Tables, rows, columns, schemas
- Primary key & foreign key
- Data types: INT, VARCHAR, DATE, DECIMAL
- NULL meaning in SQL
- SQL vs NoSQL — when to use what
- SQL dialects: MySQL, PostgreSQL, SQLite, BigQuery

Hands-on setup

- Install SQLiteOnline / pgAdmin / DBeaver
- Load a sample database (Chinook / Northwind)
- Explore tables with SELECT * FROM table
- Read an ER diagram
- Identify PKs and FKs in a schema

Day 2 — SELECT & filtering — retrieving rows

The most fundamental skill in SQL

Order of execution

`FROM` → `WHERE` → `SELECT` → `LIMIT`

SELECT clause

- SELECT specific columns
- SELECT * (avoid in production)
- Column aliases with AS
- DISTINCT — removing duplicates
- Literal values and expressions in SELECT

WHERE clause

- Comparison: =, !=, <, >, <=, >=
- BETWEEN … AND …
- IN (list) / NOT IN
- IS NULL / IS NOT NULL
- AND, OR, NOT — logical operators
- Operator precedence & parentheses
- LIMIT / TOP / FETCH FIRST n ROWS

Day 3 — Sorting, pattern matching & CASE

Ordering results and conditional column logic

Order of execution

`FROM` → `WHERE` → `SELECT` → `ORDER BY` → `LIMIT`

Sorting & patterns

- ORDER BY (ASC / DESC)
- Multi-column ORDER BY
- NULLS FIRST / NULLS LAST
- LIKE with % wildcard (any chars)
- LIKE with _ wildcard (single char)
- Chaining LIKE with AND / OR

CASE expression

- Simple CASE WHEN … THEN … END
- Searched CASE (conditional logic)
- CASE inside SELECT for derived columns
- CASE inside ORDER BY for custom sort
- Nesting CASE expressions
- COALESCE() as shorthand CASE

Day 4

Aggregate functions & GROUP BY

Summarising and grouping data for reporting

Order of execution

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Aggregate functions

- COUNT(*) vs COUNT(col) — key difference
- SUM, AVG, MIN, MAX
- COUNT(DISTINCT col) — unique count
- ROUND(value, decimals)
- NULL behaviour in aggregates
- Aggregate on filtered rows

GROUP BY & HAVING

- GROUP BY single column
- GROUP BY multiple columns
- HAVING — filter on aggregated values
- WHERE vs HAVING — critical distinction
- Grouping NULLs
- ROLLUP for subtotals (intro)

Day 5

Joins — combining multiple tables

The most important skill for working with relational data

Order of execution

FROM → JOIN → WHERE → GROUP BY → SELECT → ORDER BY

Join types

- INNER JOIN — matching rows only
- LEFT JOIN — all left + matched right
- RIGHT JOIN (rewrite as LEFT)
- FULL OUTER JOIN — all rows both sides
- CROSS JOIN — cartesian product
- Self join — table joined to itself
- Non-equi join (range / inequality)

Join best practices

- Table aliases (a.col vs b.col)
- ON clause vs USING clause
- Joining 3+ tables in sequence
- Detecting join duplicates
- Finding unmatched rows with NULL check
- Join performance: put smaller table left

Phase 2

Intermediate querying

Days 6–11

Day 6

Subqueries

Nesting SELECT statements inside other queries

Order of execution

Outer query runs → Subquery evaluated → Result returned

Subquery types

- Scalar subquery — returns single value
- Multi-row subquery with IN / NOT IN
- Subquery in FROM (derived table)
- Subquery in SELECT column
- Correlated subquery — row-by-row
- EXISTS / NOT EXISTS
- Correlated vs non-correlated distinction

When to use what

- Subquery vs JOIN — performance trade-offs
- Avoiding correlated subqueries on large sets
- Flattening subqueries into CTEs
- Debugging nested queries step by step
- Common subquery mistakes & fixes

Day 7 — Common Table Expressions (CTEs)

WITH clause — readable, reusable, chainable query blocks

Order of execution

CTE 1 defined → CTE 2 defined → Final SELECT runs

CTE fundamentals

- WITH cte_name AS (…) syntax
- Single CTE usage
- Chaining multiple CTEs
- Referencing one CTE inside another
- CTE vs subquery — readability & scope
- CTE vs temp table — when to choose

Recursive CTEs

- Recursive CTE structure: anchor + recursive member
- UNION ALL inside recursive CTE
- Traversing hierarchies (org charts, categories)
- Generating number sequences
- Cycle detection with depth limit
- Dialect differences: MySQL 8+, PostgreSQL, SQL Server

Day 8 — Window functions I — ranking & numbering

ROW_NUMBER, RANK, DENSE_RANK, NTILE, PERCENT_RANK

Order of execution

FROM / JOIN → WHERE → GROUP BY → HAVING → SELECT + OVER() → ORDER BY

Note: Window functions execute AFTER GROUP BY/HAVING and BEFORE ORDER BY — they see the full result set.

OVER() clause anatomy

- PARTITION BY — groups within window
- ORDER BY inside OVER — sort within partition
- ROWS / RANGE frame (preview)
- Difference: window function vs GROUP BY
- Multiple OVER() clauses in one query

Ranking functions

- ROW_NUMBER() — unique sequential number
- RANK() — gaps on ties
- DENSE_RANK() — no gaps on ties
- NTILE(n) — divide into n buckets
- PERCENT_RANK() — relative rank 0–1
- CUME_DIST() — cumulative distribution
- Top-N per group pattern with ROW_NUMBER

Day 9 — Window functions II — analytics & frames

LAG, LEAD, running totals, moving averages, frame clauses

Order of execution

PARTITION BY → ORDER BY in OVER → FRAME clause applied → Function computed per row

Offset functions

- LAG(col, n, default) — look back n rows
- LEAD(col, n, default) — look ahead n rows
- Period-over-period change: (this - LAG) / LAG
- FIRST_VALUE() — first in partition
- LAST_VALUE() with correct frame
- NTH_VALUE(col, n) — nth row value

Frame clauses & aggregates

- ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
- ROWS BETWEEN N PRECEDING AND CURRENT ROW
- ROWS BETWEEN CURRENT ROW AND N FOLLOWING
- SUM() OVER() — running total
- AVG() OVER() — moving average
- COUNT() OVER() — running count
- Combining multiple window functions in one query

Day 10

Date & time functions

Temporal arithmetic, extraction, truncation, formatting

Order of execution

FROM (date cols fetched) → WHERE (date filter) → SELECT (date transforms) → GROUP BY (date bucket)

Date basics

- DATE, DATETIME, TIMESTAMP, TIME types
- CURRENT_DATE, NOW(), GETDATE(), SYSDATE
- Implicit vs explicit date casting
- Date literals and dialect formats
- Storing dates as strings — the anti-pattern

Date operations

- DATEADD / DATE_ADD — add intervals
- DATEDIFF — difference between dates
- EXTRACT / DATEPART — year, month, day, hour, weekday
- DATE_TRUNC / TRUNC — floor to period
- Date range filters: BETWEEN vs >= and <
- EOMONTH — end of month
- TO_CHAR / FORMAT — display formatting
- Time zone handling basics (AT TIME ZONE)

Day 11

String & type functions

Text cleaning, transformation, type conversion

Order of execution

FROM → WHERE (string filter) → SELECT (string transforms applied)

String functions

- UPPER / LOWER — case normalisation
- TRIM / LTRIM / RTRIM — remove whitespace
- LENGTH / LEN — character count
- CONCAT / || — joining strings
- SUBSTRING / SUBSTR(str, start, len)
- LEFT(str, n) / RIGHT(str, n)
- REPLACE(str, find, replace)
- CHARINDEX / POSITION / INSTR — find substring
- SPLIT_PART / PARSENAME — split delimited strings

Type & null handling

- CAST(col AS type) / CONVERT(type, col)
- TRY_CAST — safe conversion without error
- COALESCE(a, b, c) — first non-null value
- NULLIF(a, b) — return null if a = b
- ISNULL / IFNULL / NVL
- IIF / IF() — inline conditional
- Implicit type coercion risks

Phase 3 Advanced SQL

Days 12–16

Day 12

Data cleaning & wrangling in SQL

Deduplication, profiling, pivoting, quality checks

Order of execution

Profile → Deduplicate → Transform → Validate

Deduplication

- ROW_NUMBER() OVER(PARTITION BY … ORDER BY …) for dedup
- DISTINCT vs GROUP BY for dedup
- Keeping latest / earliest record
- Finding exact vs near-duplicates
- DELETE duplicates using CTE + ROW_NUMBER

Wrangling patterns

- Data profiling: null %, min, max, distinct count
- Conditional aggregation with CASE inside SUM/COUNT
- Pivot: rows to columns using CASE
- UNPIVOT: columns to rows
- Detecting outliers with Z-score or IQR in SQL
- Cross-tab (contingency table) queries
- Standardising inconsistent category labels

Day 13 — Set operations & advanced joins

UNION, INTERSECT, EXCEPT, lateral joins

Order of execution

Query A executed → Query B executed → Set operation applied → Final result

Set operations

- UNION — combine + remove duplicates
- UNION ALL — combine keeping all rows
- INTERSECT — rows in both result sets
- EXCEPT / MINUS — rows in A not in B
- Column count and type must match
- Ordering results of UNION
- Chaining multiple set operations

Advanced join patterns

- CROSS APPLY / LATERAL JOIN
- Anti-join pattern (LEFT JOIN + WHERE IS NULL)
- Semi-join pattern (EXISTS)
- Inequality join (BETWEEN, <, >)
- Self-join for comparisons within same table
- Joining on multiple conditions
- Many-to-many join handling

Day 14 — Query optimisation & execution plans

How the engine runs your query, indexes, rewrites

Order of execution

Parser → Query planner → Optimiser → Executor → Result

Reading execution plans

- EXPLAIN / EXPLAIN ANALYZE / SET STATISTICS IO
- Seq scan vs index scan vs index-only scan
- Hash join vs nested loop vs merge join
- Cost estimation and row estimates
- Identifying bottleneck operators
- Before & after rewrite comparison

Optimisation techniques

- Clustered vs non-clustered indexes
- Composite indexes and column order
- Covering indexes — avoid table lookup
- Avoid functions on indexed columns in WHERE
- SARGability: what breaks index use
- Predicate pushdown principle
- Avoid SELECT * in production
- CTE vs temp table vs subquery — performance
- Statistics and why they go stale
- Partitioning for large tables (intro)

Day 15 — Views, stored procedures & functions

Reusable SQL objects: views, procs, UDFs, triggers

Order of execution

Object created once → Query references object → Engine resolves at runtime

Views

- CREATE VIEW syntax
- Simple vs complex views
- Updatable views and limitations
- WITH CHECK OPTION
- Materialised views — precomputed results
- Refreshing materialised views
- Views for security / row-level access

Stored procedures & functions

- CREATE PROCEDURE with parameters (IN / OUT)
- EXEC / CALL syntax
- Control flow: IF / ELSE, WHILE, BEGIN…END
- Scalar UDF — returns single value
- Table-valued function — returns a table
- Inline vs multi-statement TVF
- CREATE TRIGGER — BEFORE / AFTER, INSERT / UPDATE
- When to use procedures vs CTEs vs app code

Day 16

Advanced analytics patterns

Cohort, funnel, retention, RFM, sessionisation

Order of execution

Raw events → Sessionise / group → Aggregate cohort → Metrics output

Cohort & retention

- Cohort assignment (first event date)
- Cohort matrix: cohort month vs activity month
- Day 1 / Day 7 / Day 30 retention
- Rolling retention vs range retention
- Churn identification query
- Period-over-period active users

Funnel & RFM

- Funnel stages with CTEs + LEFT JOIN
- Drop-off rate per step
- Time-to-convert between funnel steps
- Sessionisation with time gap (LAG + CASE)
- RFM: Recency, Frequency, Monetary scoring
- NTILE-based RFM bucketing
- Customer lifetime value (LTV) query
- Attribution: first-touch vs last-touch in SQL

Phase 4 — Expert & real-world — Days 17–18

Day 17

Data modelling & analytics engineering

Star schema, dbt concepts, SCD, documentation

Order of execution

Staging layer → Intermediate models → Mart / reporting layer

Data modelling

- Star schema: fact + dimension tables
- Snowflake schema — normalised dimensions
- Grain definition — what each row represents
- Surrogate keys vs natural keys
- Bridge tables for many-to-many
- SCD Type 1 (overwrite)
- SCD Type 2 (history row)
- SCD Type 3 (previous column)

Analytics engineering (dbt-style)

- dbt model layers: staging, intermediate, marts
- ref() and source() patterns
- Incremental models — only process new rows
- Generic tests: unique, not_null, accepted_values
- SQL style guide & naming conventions
- Query documentation with comments
- Version controlling SQL (Git basics)
- Modular SQL design principles

Day 18

SQL interview prep & real-world patterns

Problem patterns, edge cases, debugging, production SQL

Order of execution

Read problem → Clarify edge cases → Write & test → Optimise & explain

Interview patterns

- Top-N per group (ROW_NUMBER pattern)
- Median calculation in SQL
- Running total and cumulative sum
- Year-over-year / month-over-month growth
- Gaps and islands problem
- Consecutive records detection
- Pivot table without PIVOT keyword
- Finding duplicate records — 3 methods
- Customers with no orders (anti-join)
- Second highest salary pattern
- Employees who earn more than their manager

Production & edge cases

- NULL in 3-valued logic — the full truth table
- Division by zero: NULLIF guard
- Date edge cases: leap years, month-end
- Integer division vs float division
- Floating point precision issues
- DISTINCT vs GROUP BY — when they differ
- Implicit type coercion traps
- Debugging SQL: top-down isolation method
- Reading & fixing someone else's buggy query
- Writing self-documenting SQL